Persistence

Java Persistence Solutions

Spring Data is a family of projects designed for **data access and repository management**.

- Spring Data Commons – Core Spring concepts underpinning every Spring Data module
- Spring Data JDBC – Repository support for JDBC
- Spring Data JPA – Repository support for JPA
- Spring Data MongoDB – Object-document support for MongoDB
- Spring Data R2DBC – Repository support for R2DBC
- Spring Data KeyValue – `Map` -based repositories for key-value stores
- Spring Data LDAP – Repository support for Spring LDAP
- Spring Data Redis – Easy configuration and access to Redis
- Spring Data REST – Exports repositories as hypermedia-driven RESTful resources
- Spring Data for Apache Cassandra – Access to Cassandra for highly available apps
- Spring Data for Apache Geode – Access to Geode for low-latency, highly consistent apps

---

Spring Data JDBC

- Standard JDBC approach, the **oldest way to access databases in Java**
- Uses **SQL statements directly**, which can be an advantage or disadvantage

Advantages:

- Improved performance due to direct database communication
- Lightweight, minimal abstraction

Disadvantages:

- Dependency on database vendor
- Changing DB (e.g., MySQL → Oracle) may require adjusting SQL for dialect differences

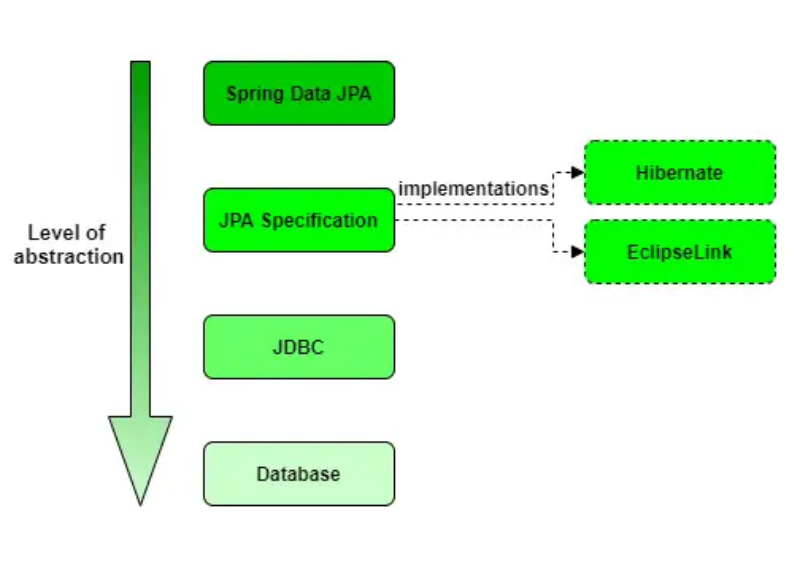
---

Spring Data JPA

- **Higher-level alternative** to JDBC using JPA (Jakarta Persistence API)
- Uses **ORM (Object-Relational Mapping)** to map classes to database tables

Key points:

- Define **entities** instead of tables
- Framework handles SQL generation and persistence
- Popular implementations: **Hibernate** (most used) or **EclipseLink** (reference)



Trade-offs:

- **Less control** over generated SQL
- **Suitable for most projects**
- Custom queries or performance-critical paths can use **JPA customizations** or **direct JDBC**

Configuration

Maven (`pom.xml`)

```
<dependencies>
    <!-- Spring Boot JPA Starter -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- PostgreSQL driver -->
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
    </dependency>

    <!-- H2 in-memory database driver -->
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

Spring Boot (`application.yml`)

PostgreSQL Configuration

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/jdbc_schema
    username: user
    password: secret
  jpa:
    open-in-view: false
    hibernate.ddl-auto: create-drop
    defer-datasource-initialization: true
  sql:
    init.mode: always
```

H2 Configuration

```
spring:
  datasource:
    url: jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1
    username: sa
    password:
  jpa:
    open-in-view: false
    hibernate.ddl-auto: create-drop
    defer-datasource-initialization: true
  sql:
    init.mode: always
  h2:
    console:
      enabled: true
```

Creating Spring Data JPA Repositories

- Repositories allow **CRUD operations without implementation code**
- Core interfaces:
  - CrudRepository – CRUD methods
  - `PagingAndSortingRepository` – Pagination and sorting
  - `JpaRepository` – JPA-specific methods (flush, batch delete)

Example repository:

```
@Repository
public interface ProductRepository extends CrudRepository<Product, Long> {
}
```

- Repository interfaces are **proxies created by Spring**, no implementation needed

---

Using Spring Data Repositories

```
@Log
@SpringBootApplication
public class App implements ApplicationRunner {
    private final ProductRepository productRepository;

    public App(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public static void main(final String... args) {
        SpringApplication.run(App.class, args);
    }

    @Override
    public void run(ApplicationArguments args) {
        productRepository.save(new Product("171f5df0-b213-4a40-8ae6-fe82239ab660", "Laptop", 2.2));
        productRepository.save(new Product("f89b6577-3705-414f-8b01-41c091abb5e0", "Bike", 5.5));
        productRepository.save(new Product("b1f4748a-f3cd-4fc3-be58-38316afe1574", "Shirt", 0.2));

        Iterable<Product> products = productRepository.findAll();
        for (Product product : products) {
            log.info(product.toString());
        }
    }
}
```

Custom Queries

Method Naming Conventions

```
List<Product> findByName(String name);
List<Product> findByWeightGreaterThan(Double weight);
List<Product> findByNameAndWeight(String name, Double weight);
List<Product> findByNameContaining(String nameFragment);
List<Product> findByWeightBetween(Double minWeight, Double maxWeight);
List<Product> findAllByOrderByNameAsc();
```

Using `@Query` Annotation

```
@Query("SELECT p FROM Product p WHERE p.name = :name")
List<Product> findProductsByName(@Param("name") String name);

@Query("SELECT p FROM Product p WHERE p.weight > :weight")
List<Product> findProductsWithWeightGreaterThan(@Param("weight") Double weight);

@Query("SELECT p FROM Product p WHERE p.name LIKE %:nameFragment%")
List<Product> findProductsByNameContaining(@Param("nameFragment") String nameFragment);

@Query("SELECT p FROM Product p WHERE p.weight BETWEEN :minWeight AND :maxWeight")
List<Product> findProductsByWeightRange(@Param("minWeight") Double minWeight, @Param("maxWeight") Double maxWeight);

@Query(value = "SELECT * FROM product ORDER BY weight DESC", nativeQuery = true)
List<Product> findAllProductsOrderedByWeightDesc();
```

Perfect! If the **DTO should not expose the entity ID**, we just remove it and keep only the fields needed by the client. Here's the revised section:

Data Transfer Objects (DTOs)

In **Spring Data JPA**, **DTOs** (Data Transfer Objects) are used to carry **data between layers** of an application, typically from the service or repository layer to the controller or client. They **decouple internal entity structure from external API**, avoid exposing sensitive fields (like IDs), and optimize queries by transferring only necessary data.

Why Use DTOs?

- **Encapsulation** – hides internal entity details from clients
- **Performance** – select only needed fields instead of entire entities
- **Flexibility** – allows combining data from multiple entities
- **Validation & Transformation** – DTOs can include computed fields or data formatted for clients

---

Example DTO (without ID)

```
public class ProductDTO {
    private String name;
    private Double weight;

    public ProductDTO(String name, Double weight) {
        this.name = name;
        this.weight = weight;
    }

    // Getters
    public String getName() { return name; }
    public Double getWeight() { return weight; }
}
```

Mapping Entities to DTOs

Manual mapping from entity to DTO:

```
public List<ProductDTO> getAllProducts() {
    List<Product> products = productRepository.findAll();
    return products.stream()
        .map(p -> new ProductDTO(p.getName(), p.getWeight()))
        .collect(Collectors.toList());
}
```

Or using **MapStruct**:

```
@Mapper
public interface ProductMapper {
    ProductDTO toDTO(Product product);
}
```

Best Practices

- Keep DTOs **immutable**
- Do **not expose entity references or IDs**
- Use DTOs for **API responses**, while entities remain internal

Resources

- Spring Data Projects
- Baeldung – Spring Data Repositories